

M12 / PROTECTED ANSWERS

M12 Module Revision Test Answers

Standalone Zero-to-Job-Ready Data Engineer 2026

Study, practise, check and explain.

A complete learning route

Start with the lesson video. Read the worked explanation beside it, pause to reproduce the example, then solve the supplied task. The books contain your study notes; keep your own work and results as evidence.

Use the protected answers after saving your first attempt. A changed retry shows what you can now do independently. There is no forced timer during learning.

One course. M00-M19. 264 lessons and 50 project phases.

Contents

Tap a topic or use the PDF bookmarks. These numbers are the actual pages in this edition.

Read this first	6
Open after a saved attempt only	6
DE12-01 Why Spark exists: driver, executors, partitions and tasks	6
Expected reasoning / control	6
Why	6
Professional boundary	6
DE12-02 PySpark setup, SparkSession and laptop-safe local mode	6
Expected reasoning / control	6
Why	6
Professional boundary	6
DE12-03 DataFrames and explicit schemas	7
Expected reasoning / control	7
Why	7
Professional boundary	7
DE12-04 Spark SQL and expression equivalence	7
Expected reasoning / control	7
Why	7
Professional boundary	7
DE12-05 Lazy transformations and actions	7
Expected reasoning / control	7
Why	8

Professional boundary 8

DE12-06 | Jobs, stages, tasks and physical plans 8

Expected reasoning / control 8

Why 8

Professional boundary 8

DE12-07 | repartition vs coalesce 8

Expected reasoning / control 8

Why 8

Professional boundary 8

DE12-08 | Aggregations and shuffle cost 9

Expected reasoning / control 9

Why 9

Professional boundary 9

DE12-09 | Joins and broadcast strategy 9

Expected reasoning / control 9

Why 9

Professional boundary 9

DE12-10 | Join correctness, fan-out and reconciliation 9

Expected reasoning / control 9

Why 10

Professional boundary 10

DE12-11 | Data skew 10

Expected reasoning / control 10

Why	10
Professional boundary	10
DE12-12 Adaptive Query Execution (AQE)	10
Expected reasoning / control	10
Why	10
Professional boundary	11
DE12-13 Caching and persistence	11
Expected reasoning / control	11
Why	11
Professional boundary	11
DE12-14 Parquet I/O and partition pruning	11
Expected reasoning / control	11
Why	11
Professional boundary	11
DE12-15 Built-in functions vs Python UDFs	12
Expected reasoning / control	12
Why	12
Professional boundary	12
DE12-16 Driver safety: collect, toPandas and bounded inspection	12
Expected reasoning / control	12
Why	12
Professional boundary	12
DE12-17 Testing and debugging Spark transformations	12

Expected reasoning / control	12
Why	13
Professional boundary	13
DE12-18 Integrated PySpark pipeline and distributed-processing checkpoint	13
Expected reasoning / control	13
Why	13
Professional boundary	13

Read this first

Open this book after saving your own attempt. Compare the reasoning and result, repair the cause of a mistake, then close the answer and try changed inputs.

Open after a saved attempt only

DE12-01 | Why Spark exists: driver, executors, partitions and tasks

Expected reasoning / control

The simulator says the driver plans/co-ordinates, executors run tasks over partitions, and local[2] is two local task threads. The Spark lab should report 18 source rows. Partition counts can differ by reader and environment; the lesson is to inspect them rather than memorise a number.

Why

Spark can parallelize because the logical dataset is split into partitions and tasks can operate on independent partitions. The driver does not personally process every row just because Python started the application. Keeping this distinction prevents later errors such as bringing all data back to the driver with `collect()`.

Professional boundary

Use the execution model to explain evidence, not as vocabulary theatre. For every Spark incident be able to state where the code runs, which operation created the partition boundary, what one output row means and which row/key/total check proves business correctness.

DE12-02 | PySpark setup, SparkSession and laptop-safe local mode

Expected reasoning / control

Your learner environment should report PySpark 4.2.0, a supported Python version and Java 17 or later. A `SparkSession` created through `common.spark_session()` should use local[2], 1g driver memory, `spark.sql.shuffle.partitions=4` and AQE true. Record the actual values rather than assuming configuration took effect.

Why

A pinned small environment separates course logic from machine noise. When every lab uses the same session builder, a plan difference is less likely to be caused by one forgotten setting. Explicitly stopping sessions also reduces the chance that multiple JVMs compete for your limited RAM.

Professional boundary

A trustworthy runtime record says what was really executed on which environment. Never turn static code inspection into a live-Spark PASS. Stable setup and honest runtime evidence make later performance observations comparable.

DE12-03 | DataFrames and explicit schemas

Expected reasoning / control

The schema should show typed date/integer/double fields instead of all strings. The resulting DataFrame keeps one row per source order and adds `gross_sales` without collecting the whole dataset into Python. The source fixture remains 18 rows.

Why

Built-in column expressions remain part of Spark's structured plan. Explicit types let Spark validate operations and choose encodings without first treating everything as Python objects. More importantly, the data contract becomes reviewable: another engineer can see what the pipeline expects before a bad batch reaches a downstream join.

Professional boundary

Prefer explicit schemas at stable ingestion boundaries and built-in expressions for transformations. Save `printSchema()` evidence, row grain and a small bounded sample; the schema alone is not proof that all business values are valid.

DE12-04 | Spark SQL and expression equivalence

Expected reasoning / control

Both routes produce four regional rows with identical totals; their sum is 25,230. The result grain is one row per region. If one route includes Pending/Cancelled rows or uses a different calculation, matching syntax style is irrelevant because business semantics differ.

Why

DataFrame and SQL expressions enter Spark's structured optimizer. Translating between them makes assumptions visible: `WHERE`/filter defines the population, `groupBy`/`GROUP BY` changes the grain, and aggregate expressions define measures. Sorting is added only to make evidence comparisons deterministic, not because DataFrames are inherently ordered.

Professional boundary

Be bilingual enough to review both APIs, but validate business equivalence with row counts, keys and totals. Matching screenshots are weak evidence; deterministic comparison of values and schema is stronger.

DE12-05 | Lazy transformations and actions

Expected reasoning / control

Before `show()`, you have a DataFrame plan object, not four already-calculated regional totals sitting in Python memory. The action produces the four totals. The exact number of jobs/stages can depend on Spark/version/operation; inspect runtime evidence rather than promising one fixed count from the source code alone.

Why

Laziness gives Spark room to optimize a whole structured plan: it can push filters, prune columns or change join strategy rather than executing each line as an isolated Python instruction. The cost is conceptual: source code order is not a timeline of physical execution.

Professional boundary

Plan first, trigger work deliberately, and record which actions exist. Do not equate Python line count with Spark work. Use explain, Spark UI/runtime metrics and independent result controls together.

DE12-06 | Jobs, stages, tasks and physical plans

Expected reasoning / control

The plan should show a distributed aggregation path with an Exchange/shuffle boundary for grouping by region. Runtime stage/task counts depend on partitions and adaptive execution; do not hard-code a universal number into your explanation.

Why

Rows with the same group key can begin in different input partitions. To compute a final per-region sum, Spark must bring compatible key groups together. That cross-partition dependency creates a stage boundary. The physical plan gives a hypothesis; runtime statistics tell you how the actual execution behaved.

Professional boundary

Use plan evidence as a map, not fortune-telling. Pair `explain('formatted')` with actual runtime metrics and result reconciliation. A faster wrong plan is still wrong.

DE12-07 | repartition vs coalesce

Expected reasoning / control

Both routes can report the requested final counts, but the repartition plan should make redistribution visible while coalesce reduction can use a narrower path. Exact plan nodes can evolve; explain the semantic distinction, not a brittle string match.

Why

Parallel work is partition-driven, so too few partitions can underuse available cores and too many tiny partitions can create scheduling/file overhead. Repartition is justified when distribution/key layout needs to change; coalesce is useful when you mainly need fewer partitions after a shrinking operation.

Professional boundary

Change partitioning for a stated reason, inspect the plan/runtime effect, and revalidate rows/keys/totals. Physical tuning must not silently change business semantics.

DE12-08 | Aggregations and shuffle cost

Expected reasoning / control

The grouped totals match the independent Python validator exactly. The Spark plan shows redistribution appropriate to a keyed aggregation. The four-row output is not evidence that only four records were processed.

Why

Filtering to Completed before grouping reduces the population from 18 to 16. Projecting only needed columns can reduce data width. Those correctness-preserving reductions occur before an expensive boundary. But do not manually micro-optimize without measuring: Catalyst can already push many filters/projections.

Professional boundary

Do not fear shuffle; make it intentional. Reduce irrelevant data early, preserve the requested grain, inspect Exchange/runtime evidence and reconcile totals after every distributed aggregation.

DE12-09 | Joins and broadcast strategy

Expected reasoning / control

With a unique dimension, the left/fact grain remains one row per Completed order: 16 rows. The plan should show a broadcast join strategy in this tiny lab. Real automatic choices depend on statistics, size, AQE and configuration, so production strategy must be verified from the actual plan.

Why

Broadcast can avoid shuffling the large relation because each executor has local access to the small lookup. The trade-off is replicated memory/network cost for the broadcast side. The engineering decision therefore depends on real size and reuse, not table naming conventions such as 'dimension'.

Professional boundary

Validate join grain and key uniqueness before tuning. Treat broadcast as a strategy backed by size/plan evidence, not a rule attached to dimension tables.

DE12-10 | Join correctness, fan-out and reconciliation

Expected reasoning / control

The bad contact join has more than 16 rows because C02/C06 duplicate. Its gross-sales sum exceeds 25,230. That is expected failure evidence. The correct repair depends on the business requirement: aggregate contacts to customer grain, choose a valid current contact by explicit rule, or keep order-contact grain and never sum order-level measures as if still one row/order.

Why

Spark's join operator obeys relational matching. If a left key has L matches and right key has R matches, that key can generate L×R joined combinations. Reconciliation catches the result even when every individual row looks plausible. This is why `count`, `countDistinct(order_id)` and `sum(gross_sales)` belong together after sensitive joins.

Professional boundary

Carry relational discipline into Spark. Every important join should have declared left/right grain, expected cardinality, unmatched policy, key checks and at least one row/key/measure reconciliation.

DE12-11 | Data skew

Expected reasoning / control

The pure-Python control reports a 0.9 HOT share. The Spark output shows HOT=900. If task-level metrics are too small to look dramatic locally, record that honestly. You have proved the skew shape, not a production slowdown.

Why

Hash partitioning sends equal keys together by design, which is exactly what grouping/join correctness needs. That means a dominant key can dominate one partition. Some remedies change execution strategy (AQE skew splitting), some change data shape (salting), and some change the business model (process giant tenants separately).

Professional boundary

Diagnose skew from distributions and runtime metrics. Choose a remedy that preserves the aggregation/join semantics, and prove both performance change and unchanged business totals.

DE12-12 | Adaptive Query Execution (AQE)

Expected reasoning / control

The course profile reports AQE true. You should be able to explain at least three adaptive capabilities and one limit: AQE reacts within supported plan structures; it does not repair wrong keys, missing filters, incorrect grain or every skew problem.

Why

Estimates before execution are imperfect. Runtime map-output statistics reveal actual partition sizes and relation sizes, allowing Spark to make safer choices. This reduces the need to guess a perfect static shuffle-partition count for every stage, while still requiring a sensible initial plan and resource envelope.

Professional boundary

Keep AQE enabled unless you have evidence-based reasons otherwise, but treat it as one layer of execution optimization. Validate data distribution, join correctness and output invariants independently.

DE12-13 | Caching and persistence

Expected reasoning / control

The business results still reconcile to 25,230. Cache should appear only after an action materializes it. After unpersist, the explicit cache is released. On this tiny dataset elapsed time is not meaningful evidence; the lab demonstrates lifecycle and reuse reasoning.

Why

Without persistence, separate actions may replay the upstream lineage. Caching stores computed partitions so reuse can avoid some recomputation. But blocks occupy memory/disk according to storage level and may be evicted. The benefit depends on cost to recompute, reuse count, size and available memory.

Professional boundary

Cache intentionally, materialize knowingly, inspect actual reuse/storage, and unpersist when the reuse window ends. Never use cache as a reflexive line in every pipeline.

DE12-14 | Parquet I/O and partition pruning

Expected reasoning / control

The read returns only rows for 2026-09-03 and the requested columns. The plan should show Parquet/file-scan information including partition filters when applicable. Real Spark/Parquet runtime evidence must be collected on the learner PC; static checks alone do not prove scan behavior.

Why

Physical layout gives the engine opportunities to avoid work, but Spark still decides a plan. Projection reduces column I/O; partition filters can avoid whole directory partitions; Parquet statistics can help skip lower-level chunks. These are distinct mechanisms and should not be collapsed into one slogan.

Professional boundary

Choose partition layout from stable access patterns and volume, keep file sizes reasonable, and verify pruning/projection in the actual plan. Logical output checks stay identical regardless of physical layout.

DE12-15 | Built-in functions vs Python UDFs

Expected reasoning / control

Both versions can produce the same uppercase values. The built-in plan stays within Spark SQL expression machinery; the Python UDF plan exposes a Python UDF evaluation boundary. This is a plan-level teaching observation, not a promise of a fixed speed ratio.

Why

Built-ins give Spark richer semantics for optimization, code generation and execution. Python UDFs require data to cross into Python workers and reduce what Catalyst can see. That overhead matters especially at scale, but correctness still comes first: use the expression that accurately represents the rule.

Professional boundary

Prefer optimizer-visible built-ins when they express the rule clearly. Use UDFs for genuine gaps, with explicit types, null/error behavior, tests and plan/runtime evidence.

DE12-16 | Driver safety: collect, toPandas and bounded inspection

Expected reasoning / control

The aggregate returns 25,230 and the bounded inspection returns at most five Row objects. No full 18-row collect is necessary even in the toy lab because we are building the habit that scales. A small result after a distributed aggregation is different from dragging raw data to Python for computation.

Why

Driver memory is separate from total cluster capacity. Distributed operations keep partitions on executors; collection centralizes them. The safest pattern is reduce/filter/aggregate/write in Spark, then bring back only a known-small diagnostic or scalar result.

Professional boundary

Keep large data distributed. Materialize on the driver only after a proven bound or when the API inherently returns a small scalar. Treat driver-memory assumptions as explicit contracts.

DE12-17 | Testing and debugging Spark transformations

Expected reasoning / control

The correct test prints PASS. The schema and four rows match. When you inject an incorrect expected value, the testing utility identifies mismatching rows rather than relying on visual inspection. This is stronger evidence than a screenshot of `show()`.

Why

Tiny tests catch transformation logic before cluster cost and data volume hide the problem. They also make refactoring safer: a change to join/partition strategy should preserve logical invariants. Runtime/performance tests are separate; a unit test cannot prove production scaling.

Professional boundary

Test transformations on tiny deterministic data, separate logical tests from performance tests, and save failed assertion evidence before repair. A plan refactor should not silently alter business invariants.

DE12-18 | Integrated PySpark pipeline and distributed-processing checkpoint

Expected reasoning / control

Source=18; Completed=16; Completed gross=25,230. Region East=5,950, North=6,380, South=6,000, West=6,900. Category Accessories=16,830, Office=8,400. Unique Completed order IDs=16. Your Spark plan/runtime evidence must be added on the learner PC; the build package supplies only static/simulator/reference validation.

Why

The checkpoint combines all four M12 layers: business semantics, distributed API correctness, physical-plan reasoning and operational safety. None substitutes for the others. A perfect explain plan with wrong totals fails; correct totals produced by collecting everything to pandas also fail the distributed-engineering objective.

Professional boundary

A Spark pipeline is mastered when you can rebuild it on changed data, prove output with independent controls, interpret the physical plan/runtime evidence, and explain resource/driver/partition safety. Save the full evidence packet; M13 then adds real Delta/lakehouse transaction semantics.